

AI ETL Engine – Personal Knowledge Base

Based on: Intelligent Data Processor Core Foundation doc + follow-up research

Section 1: The Basics – LLMs & Why Local

Q1: What is an LLM, in the context of this project?

A: An LLM (Large Language Model) is the "brain" of the Agentic/LLM Layer described in the source doc. Instead of a human writing SQL/PySpark scripts by hand for every partner's data format, the LLM reads a source schema and a destination schema and *generates* the transformation code itself. It's the piece that lets the ETL agent reason about messy, inconsistent partner data instead of relying on hardcoded rules.

Q2: Why run the model *locally* instead of just calling an API (OpenAI, Anthropic, etc.)?

A: Three main reasons, straight from the doc's framing:

- **Data privacy** – partner data (potentially sensitive, contractual, or regulated) never leaves your infrastructure.
- **Total customization** – you control the model weights, fine-tuning, and behavior instead of depending on a vendor's hosted model.
- **Cost at scale** – once hardware is in place, there's no per-token API bill for high-volume ETL runs.

The trade-off: you take on the operational burden of hosting, serving, and maintaining the model yourself.

Q3: What's the difference between a "framework" and an "engine" here?

A:

- **Framework** (LangChain, LlamaIndex) = the orchestration code layer. It's how you define tools, chain prompts together, and structure the agent's reasoning steps.
- **Engine** (Ollama, vLLM, LocalAI) = the thing that actually *runs* the model and serves inference requests. The framework talks to the engine, not the raw model file.

Think of the framework as the agent's "logic," and the engine as its "muscle."

Section 2: Local Inference Engines – Which One and Why

Q4: What are Ollama, vLLM, and LocalAI, and how are they different?

Tool	What it is	Best for
Ollama	Simple local model runner, bundles weights + config into one package	Prototyping, dev/testing, CPU-friendly work
vLLM	High-throughput, GPU-optimized production serving engine	Production workloads with concurrent requests, needs GPU
LocalAI	OpenAI-API-compatible wrapper for local models	Drop-in replacement when your code already expects the OpenAI SDK format

Q5: Why does this matter for *this* project specifically?

A: The doc's architecture has an Agentic/LLM Layer that needs to handle potentially many partner data requests. The tool you pick determines whether you're optimized for early development (Ollama) or for scaled production throughput (vLLM). Picking wrong means either overspending on GPU infra too early, or hitting a wall in production with a dev-oriented tool.

Q6: I don't have a strong GPU – which should I use?

A: Ollama or LocalAI. vLLM is built specifically for GPU throughput and performs poorly, or won't run efficiently, on CPU-only hardware. Ollama defaults to CPU-friendly quantized models, making it the natural fit for development on modest hardware.

Q7: What's the realistic path from prototype to production?

A:

1. **Prototype** the agent's logic (prompts, tool calls, schema mapping) in **Ollama** with a mid-size quantized model.
2. **Validate accuracy** against real partner data samples – does the generated SQL/PySpark actually work?
3. **Move to vLLM** for production once the model and prompt design are locked in and you need to serve concurrent requests at scale (this typically means investing in GPU hardware at that point).

Section 3: Model Selection

Q8: Which model should I actually use?

A: Depends on your constraint:

Priority	Recommended model
Lightest possible CPU load	Phi-3 mini (3.8B)
Best CPU-friendly balance	Mistral 7B (quantized)
Better reasoning, still CPU-viable	Llama 3.1 8B (quantized)
Best for schema mapping / structured data (this project)	Qwen2.5 7B-Instruct (quantized)

Why Qwen2.5 for this specific use case: it's specifically strong at structured/tabular reasoning and disciplined JSON output – which is exactly what schema mapping and transformation-logic generation require, more so than general chat ability.

Q9: Does context window size matter for feeding in database schemas?

A: Yes, significantly. If you're passing table names, columns, types, and sample rows into a prompt, a small context window will truncate mid-schema and produce broken mappings.

- Mistral 7B: ~32K context – fine for small schemas, risky for large ones.
- Llama 3.1 8B / Qwen2.5: ~128K context – much safer for real-world database schemas.

Q10: Should I actually paste the whole database into the prompt every time?

A: **No – this is a design mistake to avoid.** Even with a large context window, dumping the full schema every call is wasteful, slow, and error-prone. Instead, follow the doc's own "Dynamic Connection Tools" idea:

- Store schema metadata separately (e.g., in a vector index or lookup table).
- Give the agent a **tool** it can call to fetch *only* the relevant tables/columns for the current task.
- This keeps prompts small → faster inference → lower CPU/GPU load → more accurate results (less irrelevant noise for the model to reason through).

This directly maps to Step 2 in the doc's implementation plan ("Initialize the Agent Tools – write functions that allow the LLM to read schemas").

Section 4: Quantization – Making Models Run on Normal Hardware

Q11: What is quantization, and why do I need to know about it?

A: Quantization compresses a model's weights from high precision (16-bit) down to lower precision (e.g., 4-bit), trading a small amount of accuracy for a big reduction in memory and compute needs. Without it, even a "small" 7B model can be too heavy for a normal CPU or consumer GPU to run smoothly.

Q12: What's the difference between GGUF, AWQ, and GPTQ?

A: This trips a lot of people up – they're not interchangeable:

Format	Type	Runs on	Works with Ollama?
GGUF	Self-contained file format	CPU, consumer GPU, Apple Silicon	✔ Yes – this is what Ollama natively uses
AWQ	Quantization algorithm (stored as safetensors)	NVIDIA GPUs only	✘ No – requires lossy conversion, not recommended
GPTQ	Quantization algorithm (stored as safetensors)	NVIDIA GPUs only	✘ No – same issue as AWQ

Rule of thumb: if a Hugging Face model repo name says **GGUF**, it works in Ollama. If it says **AWQ** or **GPTQ**, it's meant for GPU-serving engines like vLLM – skip it for Ollama use.

Q13: What quantization level should I pick within GGUF?

A: Q4_K_M – it's Ollama's default for good reason. It keeps roughly 92–98% of full-precision quality while cutting memory needs by ~75%. Going lower (Q2) noticeably hurts complex reasoning tasks; going higher (Q8) rarely improves results enough to justify the extra memory, especially since that memory could instead run a *larger* model at Q4 (which usually wins).

Q14: How do I actually pull the right model in Ollama?

A:

```
ollama pull llama3.1:8b-instruct-q4_K_M
# or, for this project's recommended model:
ollama pull qwen2.5:7b-instruct-q4_K_M
```

Section 5: Architecture — Tying It Back to the Doc

Q15: How does all this map onto the doc's 3-tier architecture?

A:

- **Orchestration Layer** (Airflow/Kestra) — schedules when the ETL jobs run. Not affected by model/engine choice.
- **Agentic/LLM Layer** — this is where Ollama/vLLM + your chosen model (e.g., Qwen2.5 7B) live. This layer generates the SQL/PySpark transformation code.
- **Execution & Storage Layer** (Databricks/Snowflake/PySpark) — where the generated code actually runs against real data.

Q16: Where does validation fit in, and why does it matter more with local models?

A: Step 4 of the doc ("Add Validation") is critical, and even more so with smaller/quantized local models: they're more resource-friendly but less reliable than larger hosted models at getting complex transformation logic exactly right on the first try. A schema validator or data quality checker catching bad LLM output *before* it commits to the database is what makes a smaller local model viable for production — it's a safety net for the accuracy you're trading away by staying lightweight.

Q17: If building this from scratch is too much, what's the fallback?

A: The doc lists enterprise platforms with built-in AI-augmented ETL as a lighter-weight alternative to a fully custom build:

- **Databricks** (Lakeflow + `ai_extract` / `ai_classify`)
- **Matillion & Airbyte** (AI-assisted schema mapping)
- **Informatica** (AI-powered mapping + natural language pipeline generation)

These trade customization/data-privacy control for much faster time-to-value.

Quick Reference Cheat Sheet

- **Dev/prototype, no GPU** → Ollama + Qwen2.5 7B (Q4_K_M, GGUF)
- **Production, has GPU, needs throughput** → vLLM + larger model, AWQ/GPTQ format

- **Don't** paste whole DB schemas into prompts — build a schema-lookup tool instead
- **Don't** try to load AWQ/GPTQ models into Ollama — look for "GGUF" in the repo name
- **Always** add a validation step before committing LLM-generated transformations to the database